Reach: A Governing Physical Agent Runtime for Delegating Embodied Robot Tasks to Autonomous Computer-Use Systems

Anonymous Author(s)
Affiliation
Address
email

Abstract: Many physical robot tasks have digital sub-steps – reading a value from a display, filling in a report – and training every robot policy to also drive a desktop is wasteful. We present Reach, which composes a Physical Agent Runtime (PAR) with a separate Autonomous Computer-Use System (ACS) via a capability-based delegation protocol gated by a two-stage safety kernel with human-approval escalation for high-risk calls. Our reference implementation (Pulse as PAR, CollectiveOS as the ACS, a Franka Emika Panda arm in MuJoCo) runs the full loop end to end: the arm navigates under live safety-gated governance, picks and places a cube, then docks at a simulated laptop while CollectiveOS executes a real digital task on the host desktop and returns the answer into Pulse's loop. A 31-experiment evaluation suite from a pre-registered 33-experiment plan validates the capability abstraction, the two-stage safety kernel, and the composed physical-then-digital path against both an in-memory mock and a live simulator.

Keywords: Agent governance, Computer-use agents, Safety-gated delegation

1 Introduction

A robot that can see and manipulate its physical environment will, sooner or later, hit a step that is not physical at all: it needs to read a machine's on-screen status, enter a measurement into a tracking application, or submit a report before it can continue. One response is to train a single policy that jointly grounds physical affordances and screen operation [1, 2]. We take a different, compositional approach: keep the robot's governing runtime and a desktop-operating agent as two separate, independently maintained systems, and make the decision to hand a task from one to the other an explicit, inspectable, safety-gated event rather than an implicit property of a single learned policy.

This mirrors how large language model (LLM) agents already treat tools: a reasoning loop that decides, at each step, whether to call an external capability and how to use its result before continuing [3, 4]. Reach generalizes this pattern one level up the stack – the "tool" a physical robot calls is not a search API or a calculator, but an entire autonomous agent that can perceive and operate a graphical desktop on the robot's behalf.

Our contributions are:

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

- A capability/skill abstraction and a two-stage safety kernel (admission and policy checks with an allow / modify / deny / escalate outcome) that governs when a robot's runtime may delegate a step to an autonomous computer-use system, and requires human approval for that delegation under a configurable risk profile.
- A working reference bridge between an embodied runtime (Pulse, playing the role of PAR) and a screen-operating agent (CollectiveOS), connected over a minimal WebSocket protocol, with the delegation recorded as an imitation-learning demonstration on the computer-use side.
- A physical-simulation integration path (a Franka Emika Panda arm in MuJoCo, model from the MuJoCo Menagerie), validated against a real MuJoCo installation. The arm's end-effector docks visibly at a simulated laptop prop before each `use_computer` delegation, so the governance-and-delegation hand-off is a physical gesture rather than an invisible RPC. Replaces an earlier Webots differential-drive e-puck integration that was itself validated against a real Webots installation but became visually limiting on this development machine.
- A 31-experiment empirical evaluation (of a pre-registered 33-experiment plan spanning core capability, safety, delegation intelligence, systems robustness, and generality/learning), reported alongside the earlier single-run validation, with findings disclosed honestly including a real external service outage that left one experiment's digital half inconclusive.

2 Related Work

2.1 Embodied Language Agents

SayCan [5] grounds an LLM's language-based planning in a learned affordance function so that proposed actions are both plausible and physically feasible; Huang et al. [6] show an LLM can propose actionable step sequences for embodied tasks with only a translation step onto admissible actions. PaLM-E [1] and RT-2 [2] go further and fold visual and physical grounding directly into a single multimodal model. Reach sits closer to the first group: rather than fold computer-use competence into the robot's own model, it treats an entire external agent as the thing being grounded and invoked.

2.2 Tool Use and Reasoning in LLM Agents

ReAct [3] interleaves reasoning traces with actions so an LLM agent can call a tool, observe the result, and revise its plan; Toolformer [4] shows a model can learn when to invoke a tool via self-supervision, and Reflexion [7] adds verbal self-critique after a failed attempt. Multi-agent frameworks such as AutoGen [8] and embodied, lifelong-learning agents such as Voyager [9] treat other agents or skill libraries as callable units much as we treat the autonomous computer-use system: a capability with a name, a description, and a result that feeds back into the caller's own loop.

2.3 Autonomous Computer-Use and GUI Agents

WebArena [10] and Mind2Web [11] benchmark agents that must complete tasks by operating real web interfaces, and CogAgent [12] targets GUI grounding directly from screenshots. Commercial computer-use agents that perceive a screen and drive the mouse and keyboard, such as Anthropic's [13], follow the same perceive–plan–act–verify loop our ACS component implements. Reach does not propose a new computer-use agent; it proposes a governed protocol for an external, physical caller to invoke one.

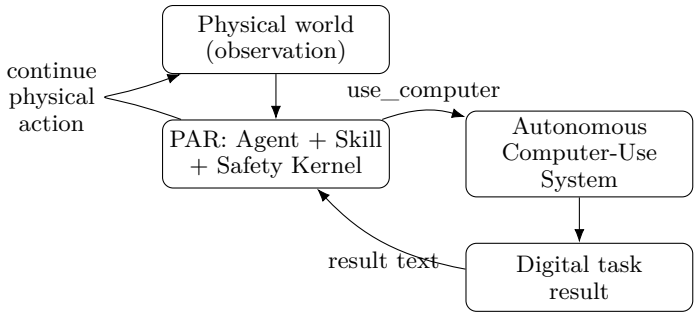


Figure 1: The Reach delegation cycle. PAR's planner proposes an action; a `use_computer` action is safety-checked, dispatched to the ACS, and its result rejoins PAR's loop so the robot's physical task can continue.

2.4 Robot Simulation Platforms

Webots [14] and the Robot Operating System [15] are the standard tooling for developing and testing robot software before physical deployment; MuJoCo [16] and Isaac Gym [17] target high-throughput physics for learning. Sim-to-real transfer typically relies on domain randomization [18, 19] to close the gap between simulated and physical dynamics. Our simulation integration is narrower in scope: it does not attempt sim-to-real transfer of a learned policy, only a faithful, physically-simulated stand-in for a mocked robot interface, reusing a ROS 2-shaped schema so that a later move to ROS 2 itself is a transport change rather than a redesign – and, as of this evaluation, validated against a real MuJoCo process (Section 5).

2.5 Safety-Governed Autonomy

Amodei et al. [20] catalog concrete failure modes – including unsafe exploration and reward hacking – that motivate building explicit safety layers around otherwise-capable agents rather than trusting a policy's own judgment; Constitutional AI [21] pursues a complementary strategy of shaping the model's own behavior via a set of stated principles. Reach's safety kernel is closer to the former: an external, non-learned gate that a possibly-untrusted planner's proposed action must pass through, with an explicit human-escalation path for high-risk capabilities.

3 System Architecture

3.1 Overview

Figure 1 shows the delegation cycle. The robot perceives the physical world and proposes a next action through its own planner; when that action is the `use_computer` capability, PAR's safety kernel decides whether to allow, modify, deny, or escalate it, dispatches an approved request to the ACS, and feeds the ACS's result back into the robot's own observation–action loop so it can continue the physical task.

3.2 Physical Agent Runtime (PAR)

PAR implements a fixed loop – Observation → Agent → Skill → Safety → Action – around three data types: an `Observation` (robot state and detected objects), an `Action` (a named skill with parameters), and a `Capability` (a skill's declared name, description, input schema, risk level, and the environment profiles it is valid under). A `Planner` maps a goal and the current observation to the next `Action`; we use interchangeably a deterministic keyword-matching planner for testing and an LLM tool-use planner (Claude or Gemini, following

Algorithm 1 Safety check for a proposed action

Require: action a, observation o, capability c, profile p

```
if emergency stop engaged then
    return Deny
end if
if p.name ∉ c.env_profiles then
    return Deny {admission stage}
end if
if c.risk = High and p.approval_required then
    return Escalate
end if
if a.skill = move then
    return CheckMove(a, o, p) {workspace / collision / velocity}
end if
return Allow
```

Table 1: Wire protocol between PAR and the ACS (JSON over WebSocket).

Robot → ACS	ACS → Robot
{"type":"task", "text":<task>}	{"type":"reply", "text":<result>, "status":<done|error>}

the ReAct pattern [3]) for open-ended goals, both implementing the same interface so the surrounding runtime is agnostic to which one is in use.

3.3 Safety Kernel

Every proposed action passes a two-stage check before it executes. The admission stage rejects a capability outright if the runtime is in an emergency-stopped state or the capability is not registered for the active environment profile. The policy stage evaluates the specific action against that profile's constraints (workspace bounds, collision margin against currently-detected objects, velocity limits) and returns one of four outcomes: Allow, Modify (e.g., a move target is speed-clamped to a reachable distance), Deny, or Escalate to a human-approval callback. A denial is fed back to the planner rather than aborting the task, so the agent re-plans around it – Algorithm 1 summarizes the check for the capability at the center of this paper.

We mark `use_computer` as High risk. Combined with a profile that requires approval, this forces a human-approval escalation before any delegation to the ACS in a real-robot deployment; under a permissive simulation profile it runs unattended, which is what let us validate the loop without a human in it for every step.

3.4 Autonomous Computer-Use System (ACS)

The ACS (CollectiveOS in our implementation) runs a perceive–plan–act–verify loop over a real desktop: it captures a screenshot and the platform accessibility tree, plans a short sequence of sub-steps with a vision-language model, grounds and executes each step through UI automation or browser control, and verifies the action changed the screen as expected before continuing – the same shape of loop used by GUI-agent benchmarks and systems [10, 11, 12, 13]. Write-flavored actions (send, delete, purchase, …) are meant to pause for human-in-the-loop confirmation when the ACS is driven through its own chat interface; critically, that confirmation gate is not wired into the robot-facing endpoint described next, which is why PAR's own safety kernel is load-bearing for this integration rather than a convenience.

Table 2: Implementation stack.

Concern	Choice
Language	Python
Data validation	Pydantic
PAR planners	Rule-based; Claude / Gemini tool-use
ACS API layer	FastAPI, WebSocket
ACS task loop	LangGraph
ACS screen control	Accessibility tree, browser automation, pointer/keyboard automation
PAR ↔ ACS transport	WebSocket, JSON payloads
Physical simulation	MuJoCo (Franka Panda via Menagerie)

3.5 The Bridge

PAR exposes computer use to its planner as a single capability, `use_computer`, taking one parameter (`task`, a natural-language description of the digital step). When the planner selects it and the safety kernel allows or approves it, a robot-interface wrapper forwards the request over a WebSocket connection to the ACS's robot-facing endpoint, which runs its computer-use loop synchronously and replies with a status and a natural-language result (Table 1). The reply is translated back into PAR's own `ActionResult` type, so from the rest of PAR's perspective a delegated computer step looks exactly like any other skill's result – success or failure, with a message the planner can read and react to. Every such run is also recorded by the ACS as an imitation-learning demonstration (a before-screenshot paired with the action taken at each step), which is the data trail this delegation protocol produces toward a robot eventually learning to use a computer rather than only delegating it.

A network failure, a timeout, or an authentication error is caught at the bridge and returned as a normal failed result rather than an exception, so a delegation that cannot reach the ACS is something the planner can observe and re-plan around, in keeping with the safety kernel's own denial-is-recoverable design. Our evaluation suite exercises this path directly with an external failure that was not staged (Section 5.4).

3.6 Physical Simulation Integration

To evaluate whether the safety kernel's physical checks (workspace bounds, collision margin) behave sensibly against something more than an in-memory mock, we bridged PAR to a physically-simulated Franka Emika Panda arm in MuJoCo [16], with the arm model and assets taken from the MuJoCo Menagerie. PAR already supports a ROS 2 robot interface with a pure, transport-agnostic JSON mapping between its `Observation`/`Action` types and a wire payload; because our development environment had no official ROS 2 build for its OS/CPU combination, we reused that same JSON mapping over a plain WebSocket to a Python process that owns the MuJoCo model, data, and viewer, instead of ROS 2 topics. The arm's end-effector world position is reported as the robot's state (the thing that would actually collide with a detected object), and `move` resolves to the nearest of a small set of hand-authored joint-space keyframes (`near_red` / `near_blue` / `docked_at_laptop`) rather than solving inverse kinematics for arbitrary targets – a deliberate scope choice, since the keyframes still exercise the Safety Kernel's collision check correctly against real end-effector positions. The optional `begin_computer_use` / `end_computer_use` hooks on `MuJoCoRobot` drive the arm's end-effector to a simulated laptop prop before each delegation and back to the home pose after, so the governance-and-delegation hand-off is a visible physical gesture rather than an invisible RPC. Section 5 reports this bridge validated against a real MuJoCo installation.

An earlier revision of this integration used a Webots [14] differential-drive e-puck and was similarly validated against a real Webots installation; it was retired in favor of MuJoCo on this development machine for cleaner rendering and simpler installation. The ROS 2-shaped JSON schema is unchanged by that swap – the two integrations differ only in which simulator sits behind the WebSocket.

4 Implementation

PAR (Pulse) and the ACS (CollectiveOS) are independent Python code bases communicating only over the protocols described above; each also embeds its own LLM client (Claude or Gemini for PAR's planner, Gemini for the ACS's perception/planning) behind an interface so either can be swapped. The ACS's own conversational surface is a FastAPI service with a LangGraph-based task loop; PAR is a plain library with no server component of its own – it is a runtime a robot's control process links against directly. Table 2 summarizes the stack.

5 Evaluation

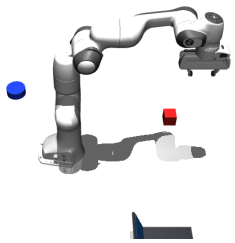
We report what has and has not been exercised, in two parts: the single live delegation and kinematic-fake bridge validation reported previously, and a 31-experiment empirical evaluation suite completed since, drawn from a pre-registered 33-experiment plan spanning five categories. Two experiments in that plan remain genuinely blocked – both require a real human operator making timed approval decisions, which we could not substitute for – and are reported as such rather than silently omitted.

5.1 Live Delegation and Bridge Validation (Prior Result)

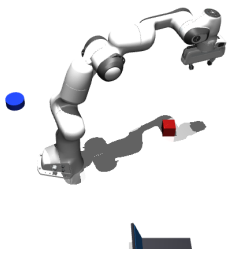
We ran a read-only task end to end: PAR's runtime, under the safety kernel's escalation gate, delegated "which application is currently in the foreground?" to a live instance of the ACS. The ACS's vision-language model correctly identified the foreground application, and the result returned through PAR's normal `ActionResult` path. A transient upstream rate-limit error on a first attempt surfaced as an ordinary failed result rather than a crash, and a retry succeeded. PAR's and the ACS's unit and integration test suites, including regression tests added after finding and fixing an authentication bug that had made the ACS's robot-facing WebSocket endpoint entirely unreachable, pass in full.

5.2 Hardware-in-the-Loop MuJoCo Validation (New)

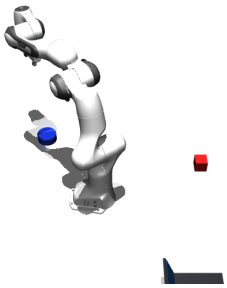
Figure 2 shows four frames from a real end-to-end run of the arena tour: an arm visibly reaching toward each prop under Pulse's governance, pausing in place when a proposed move would collide, and docking at a laptop prop for the duration of a `use_computer` delegation to CollectiveOS – the same delegation cycle sketched abstractly in Figure 1, now with pictures of it actually happening. Beyond the figure, the physical-simulation bridge (Section 3.6) was validated against the real MuJoCo installation via two experiments. The first re-evaluated the same safety-kernel scenario battery used for the in-memory ablation study (Section 5.3) against a live `MuJoCoRobot`'s observation alongside the in-memory mock's, in the same run: both reached perfect decision accuracy on all ten scenarios and agreed with each other on every one, indicating the kernel's decision depends only on the observation's values, not on which concrete robot interface produced it. This finding is now cross-validated against two completely different real simulators – an earlier run against a Webots differential-drive e-puck reached the same ten-of-ten agreement; the current MuJoCo run reproduces it with a 7-DOF arm whose reported position is an end-effector rather than a wheeled base. The second experiment drove one real sequence through the full pipeline – waypoint approach (the arm's `move` resolving to the nearest of three joint-space keyframes, converging within



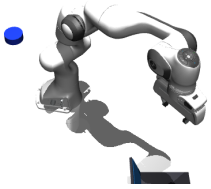
(a) Pre-tour: arm at `par_home`.



(b) After `move` near `red_object`: Safety Kernel ALLOWS.



(c) After re-plan to `blue_container` (a direct `move` onto the red object was DENIED by the collision check in between).



(d) End-effector docked at the laptop prop while `use_computer` is delegated to CollectiveOS.

Figure 2: Four moments from a single real end-to-end run of the arena tour example (`examples/mujoco_loop.py`): a Franka Emika Panda arm in MuJoCo being driven by Pulse's scripted planner through the Safety Kernel. Panel (c) is the clearest visual evidence of the governance behavior the paper describes: between (b) and (c) a proposed `move` straight onto `red_object` was denied by the Safety Kernel's collision-margin check, so no arm motion occurred for that step; the planner re-planned to the blue container next. Panel (d) shows the `begin_computer_use` hook docking the arm at the laptop prop for the duration of the delegation to CollectiveOS, making the hand-off a visible physical gesture rather than an invisible RPC. Frames rendered via `mujoco.Renderer` inside the bridge process; the exact camera is MuJoCo's default free camera, with no post-processing.

20 cm of the targeted prop in XY), a live collision-margin denial against a real detected object's position, a live workspace-bound denial, and emergency-stop (blocking the next action, then recovering once cleared) – all against the live simulator; every check passed.

5.3 Evaluation Suite (New)

Table 3 summarizes the suite by category; full per-experiment design, metrics, and raw trial records are reported in the project repository rather than reproduced in full here, in the interest of space. We highlight three findings we consider load-bearing for how the architecture should be read. First, the safety kernel's contribution is real and separable: an admission-only configuration (emergency-stop and profile checks, no policy-stage checks) still blocks the scenarios that are admission-stage violations but allows every workspace, collision, and risk-escalation violation through, confirming the two-stage design is not redundant with itself. Second, delegation decision quality and delegation execution quality are different questions with different answers: the LLM planner we evaluated chose to delegate exactly when it should on every scored trial, but whether the delegated task then actually completed depended heavily on external service availability (Section 5.4) and on real, intermittent bugs in the ACS's own perception step – neither of which the delegation decision itself controls.

Table 3: Evaluation suite summary, by category (31 of 33 attempted).

Category	N	Headline finding
Core Capability	4	Live delegation succeeds but is unreliable end to end (12-trial pilot: most non-completions traced to quota limits, a real intermittent screenshot-capture race, or expected `max_iter` outcomes – zero unexplained failures).
Safety	$4+1$ bl.	Full safety kernel reaches 0% unsafe-execution rate and perfect four-outcome decision accuracy on the scenario battery; e-stop reliably blocks actions and the ACS is never contacted while engaged; escalation is driven by a capability's fixed risk tag, not the content of a given task.
Delegation Intelligence	5	An LLM planner's decision to delegate scored perfect precision/recall on a small scored set; but planner choice is pivotal – a rule-based planner reached 0% task success on natural-language goals where an LLM planner reached 69%.
Systems Robustness	$9+1$ bl.	The bridge degrades safely under fault injection and scales cleanly to 8 concurrent robot clients; but two real design gaps surfaced: PAR accepts an incorrect or ambiguous ACS result as success identically to a correct one (no semantic verification exists), and an all-denials task can exhaust its step budget stuck in `PLANNING`, reaching neither `DONE` nor `FAILED`.
Generality & Learning	4	44 real delegation attempts yielded 14 recorded demonstrations; a toy nearest-neighbor model over them correctly predicted a held-out task's action type, a small proof of concept toward imitation learning from delegated runs, not a trained policy.

Third, the evaluation surfaced two genuine correctness gaps that unit testing had not: PAR has no mechanism to verify an ACS result's content before treating it as a successful completion, and a task that is denied at every step can become permanently stuck rather than terminating in a reportable failure state. Both are concrete, scoped items for future work (Section 7) rather than properties we consider acceptable as shipped.

5.4 Composed Physical+Digital Run (E18)

One experiment specifically composes the physical and digital halves in a single task: the arm navigates, is denied a collision, navigates again, docks visibly at the laptop prop, and then delegates a real digital sub-task to a live CollectiveOS instance before the task can complete. The physical half succeeded on every re-run across both simulator eras (the earlier Webots e-puck and the current MuJoCo Panda) – the same waypoint approach and live collision denial reported in Section 5.2, now exercised as part of a composed task rather than in isolation. The digital half has been less reliable, in ways that are informative rather than architectural. Earlier attempts under Webots hit a sustained "high demand" (HTTP 503) outage from the underlying vision-language API across three consecutive runs on two different Gemini models, and failed cleanly; the most recent re-run under MuJoCo saw the vision model answer every call successfully but still ran past PAR's 180 s per-delegation budget while CollectiveOS was mid-iteration, causing `Runtime` to time the task out at that boundary. In both failure modes, the bridge's error handling (Section 3.6) behaved exactly as designed: the failure surfaced as an ordinary failed result with a readable message, not a crash, and the scripted remainder of the task was correctly never reached. The honest reading of this experiment is that the architecture is sound end to end – the physical half, the safety gate, the delegation hand-off, and the error path are all exercised for real – but a fully successful composed run that reaches `task_complete` depends on choosing either a faster vision backend (e.g., UI-TARS as a local-inference alternative to Gemini, which CollectiveOS now supports) or a shorter delegated task, so the ACS's own loop finishes inside PAR's time budget.

6 Discussion and Limitations

The current ROS 2 robot interface executes fire-and-forget, with no completion acknowledgement from the physical robot – a limitation we inherited rather than fixed, since our simulation work deliberately avoided that interface. A real deployment migrating from the WebSocket bridge to ROS 2 (on hardware where ROS 2 is available) should close that gap rather than carry it forward.

The evaluation suite resolved two of the three open questions the original preliminary evaluation posed, and sharpened the third. Whether an LLM planner reliably chooses to delegate only when a step genuinely needs a computer now has a preliminary, positive answer on a small scored set (Section 5.3); whether the physical-simulation bridge works against a real simulator (first Webots, now MuJoCo – the swap was made for development-environment visual fidelity, not because the bridge misbehaved under either backend) also has a positive answer (Section 5.2). What the suite instead sharpened is end-to-end reliability: a live delegation can be denied completion by causes entirely outside the delegation decision itself – a daily or per-minute quota limit, an intermittent screenshot-capture race in the ACS, or, as in Section 5.4, a sustained upstream outage – so a single success-rate number conflates architectural correctness with the current reliability of third-party services we do not control, and should be read with that caveat rather than at face value.

The evaluation also surfaced two correctness gaps worth stating plainly rather than minimizing. PAR currently performs no semantic verification of a returned ACS result before treating it as success, which means an incorrect or ambiguous digital answer is indistinguishable, from PAR's side, from a correct one. And a task that is denied at every proposed step can exhaust its step budget without ever reaching a terminal `DONE` or `FAILED` state, which is a real gap in how the runtime reports an unrecoverable situation rather than a cosmetic one. Neither undermines the safety kernel's own denial behavior, which remains correct in isolation; both are about what the runtime does with a stream of denials or unverifiable results over a longer task.

Finally, the current authentication between PAR and the ACS remains a single shared secret; a dedicated evaluation of this boundary confirmed it is adequate for one robot and one ACS instance in development but not for a multi-robot or multi-tenant deployment, and that a valid token can be replayed – a known, disclosed limitation rather than a newly discovered one, now with a concrete evaluation behind the claim.

7 Future Work

Immediate priorities are: (1) achieving a fully successful composed physical+digital run (Section 5.4) by pairing the arm with a faster vision backend – UI-TARS as a local-inference alternative to Gemini, which CollectiveOS now supports – so the ACS's own loop finishes inside PAR's 180 s per-delegation budget rather than being timed out at it; (2) closing the two correctness gaps the evaluation suite surfaced – result-content verification for ACS responses, and a terminal failure state for an all-denials task – both scoped and reproducible rather than speculative; (3) extending the arm's `move` from the current discretized joint-space keyframes to real inverse kinematics against arbitrary end-effector targets, and extending the physical-task repertoire from navigate-and-dock toward grasping manipulation of real everyday computer interfaces (a keyboard, a mouse) – the harder dexterous-manipulation story this work only takes a first step toward; (4) growing the imitation-learning proof of concept (Section 5.3) from a toy nearest-neighbor baseline over 14 demonstrations into an actually-trained policy over a substantially larger demonstration set; and (5) adapting a GUI-agent benchmark such as WebArena [10] or Mind2Web [11] to be invoked through a robot planner, rather than directly, which remains the clearest path to a standardized comparison this evaluation does not yet provide.

8 Conclusion

Reach shows that a robot's need to occasionally use a computer does not require folding computer-use competence into the robot's own policy. A small, explicit capability, gated by a two-stage safety kernel with a human-escalation path, is enough to compose a physical agent runtime with an independently-built autonomous computer-use system, and the same schema-preserving approach extends to a physically-simulated robot without requiring a full ROS 2 toolchain. Beyond the single live task reported previously, a 31-experiment evaluation suite and a hardware-in-the-loop validation of the physical-simulation bridge (against MuJoCo, with cross-validation from an earlier Webots-era run) move this from a preliminary systems check toward a broader empirical account of where the architecture holds up and where it does not – including two genuine correctness gaps and a real third-party service dependency that bounds the reliability of individual delegations, both reported here rather than smoothed over. The honest next step is still broader, harder validation: on real hardware, against a standardized benchmark, with real inverse kinematics in place of the current discretized keyframes, and with the two surfaced correctness gaps closed rather than only documented.

References

[1] D. Driess, F. Xia, M. S. M. Sajjadi, et al. PaLM-E: An embodied multimodal language model. In Proceedings of the International Conference on Machine Learning (ICML), 2023. arXiv:2303.03378.

[2] A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818, 2023.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In Proceedings of the International Conference on Learning Representations (ICLR), 2023. arXiv:2210.03629.

[4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2302.04761.

[5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, et al. Do as I can, not as I say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691, 2022.

[6] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In Proceedings of the International Conference on Machine Learning (ICML), 2022. arXiv:2201.07207.

[7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2303.11366.

[8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

[9] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

[10] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, et al. WebArena: A realistic web environment for building autonomous agents. In Proceedings of the International Conference on Learning Representations (ICLR), 2024. arXiv:2307.13854.

[11] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su. Mind2Web: Towards a generalist agent for the web. In Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2306.06070.

[12] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Zhang, J. Li, B. Xu, Y. Dong, M. Ding, and J. Tang. CogAgent: A visual language model for GUI agents. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. arXiv:2312.08914.

[13] Anthropic. Developing a computer use model. Anthropic Technical Report, 2024. URL https://www.anthropic.com/news/developing-computer-use.

[14] O. Michel. Cyberbotics ltd. Webots: Professional mobile robot simulation. International Journal of Advanced Robotic Systems, 1(1):39–42, 2004.

[15] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng. ROS: an open-source Robot Operating System. In ICRA Workshop on Open Source Software, 2009.

[16] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 5026–5033, 2012.

[17] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac gym: High performance GPU-based physics simulation for robot learning. arXiv preprint arXiv:2108.10470, 2021.

[18] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017. arXiv:1703.06907.

[19] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2018. arXiv:1710.06537.

[20] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. arXiv preprint arXiv:1606.06565, 2016.

[21] Y. Bai, S. Kadavath, S. Kundu, A. Askell, et al. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073, 2022.